

INSTITUTE OF SPACE TECHNOLOGY
KICSIT, Kahuta Campus



APP DEVELOPMENT

Code Snippets

SUBMITTED TO:

SIR UZAIR HASSAN

SUBMITTED BY:

MARIA SALEEM
242201075

Department of Computer Science,
KICSIT Kahuta.

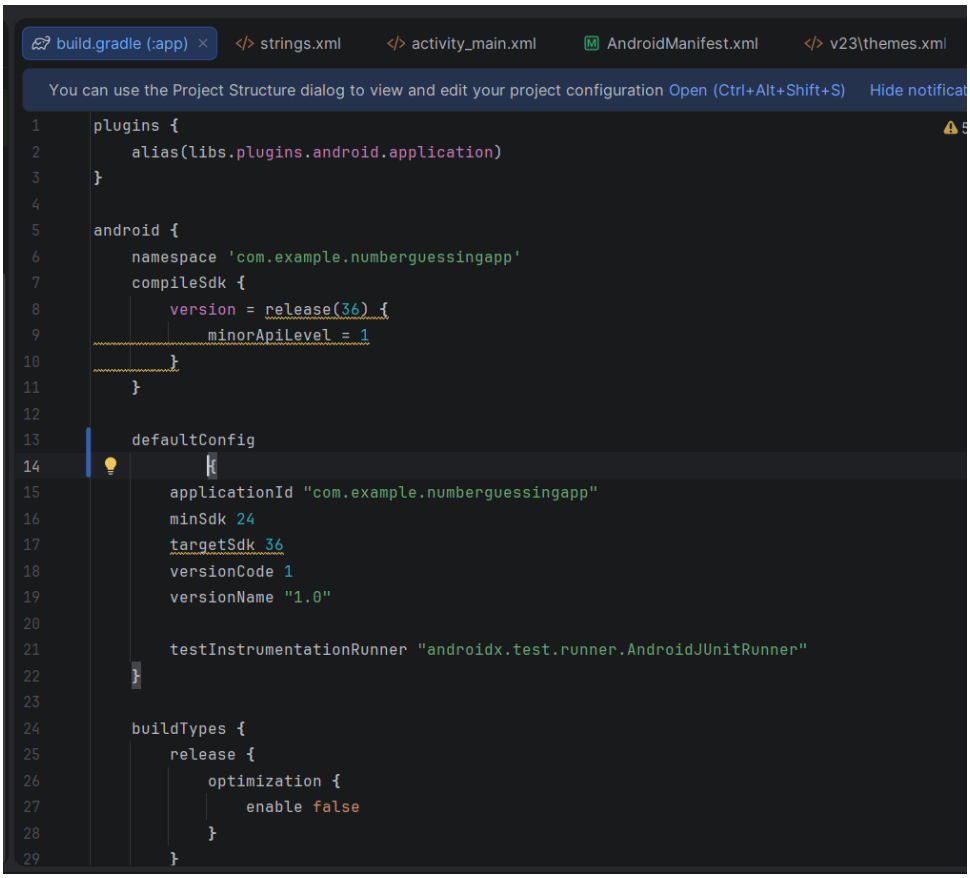
Table of Contents

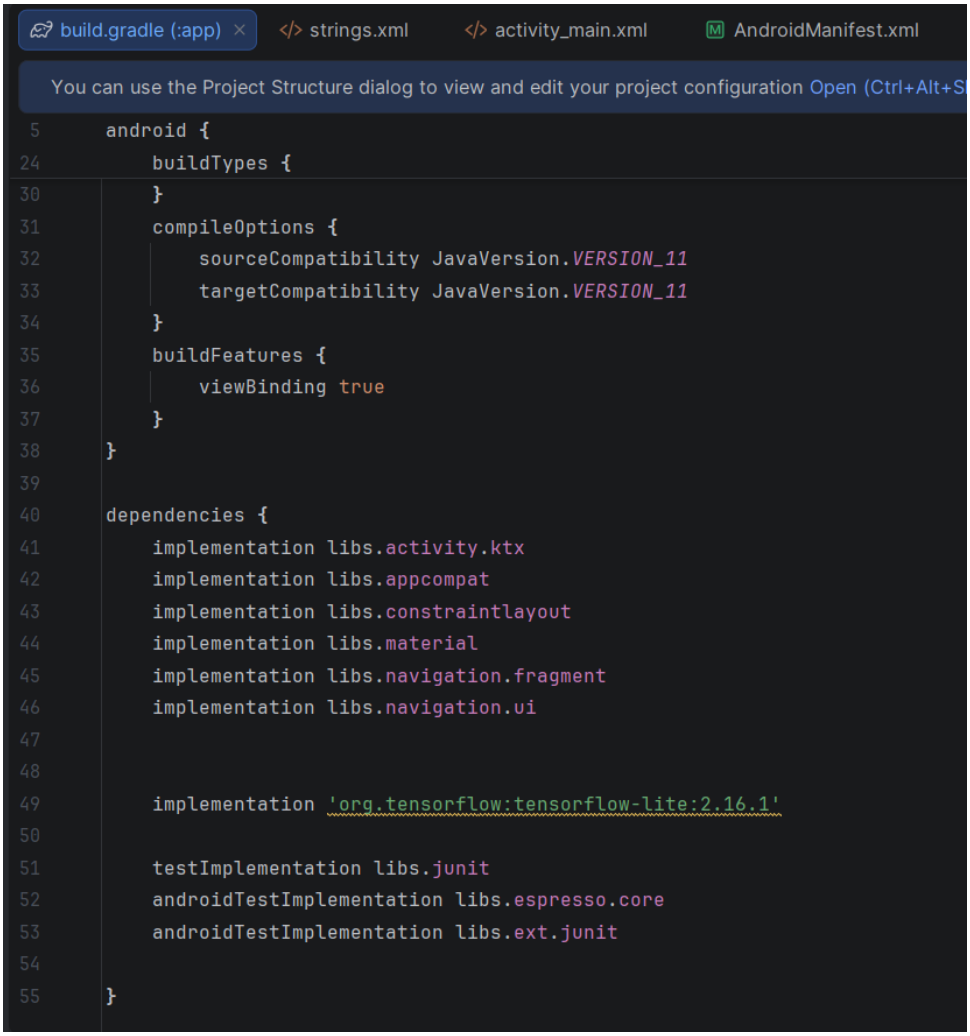
- 1. Introduction
- 2. Gradle Dependencies and Configuration
- 3. Android Manifest Configuration
- 4. Splash Screen Implementation
- 5. User Login Functionality
- 6. User Signup Functionality
- 7. Main Activity Initialization
- 8. Drawing View Implementation
- 9. TensorFlow Lite Model Loading
- 10. Model Inference and Prediction
- 11. Output Processing
- 12. Conclusion

1 Code Snippets and Explanations

1.1 Gradle Dependencies and Configuration

1.2 Gradle Dependencies and Configuration





Explanation:

Gradle configuration manages the libraries and dependencies required by the Android application. The TensorFlow Lite dependency enables the application to load and execute the trained digit recognition model. Other dependencies support the application's functionality and development environment.

1.3 Android Manifest Configuration

build.gradle (:app) | strings.xml | activity_main.xml | AndroidManifest.xml | v23\themes.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="Number guessing app"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.NumberGuessingApp">

        <!-- Splash Screen Activity -->
        <activity
            android:name=".SplashActivity"
            android:exported="true"
            android:theme="@style/Theme.App.Starting">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <!-- Login Activity -->
        <activity
            android:name=".LoginActivity"
            android:exported="false" />

        <!-- Signup Activity -->
        <activity
```

Text | Merged Manifest

```xml
        <!-- Login Activity -->
        <activity
            android:name=".LoginActivity"
            android:exported="false" />

        <!-- Signup Activity -->
        <activity
            android:name=".SignupActivity"
            android:exported="false" />

        <!-- Main Activity -->
        <activity
            android:name=".MainActivity"
            android:exported="false" />

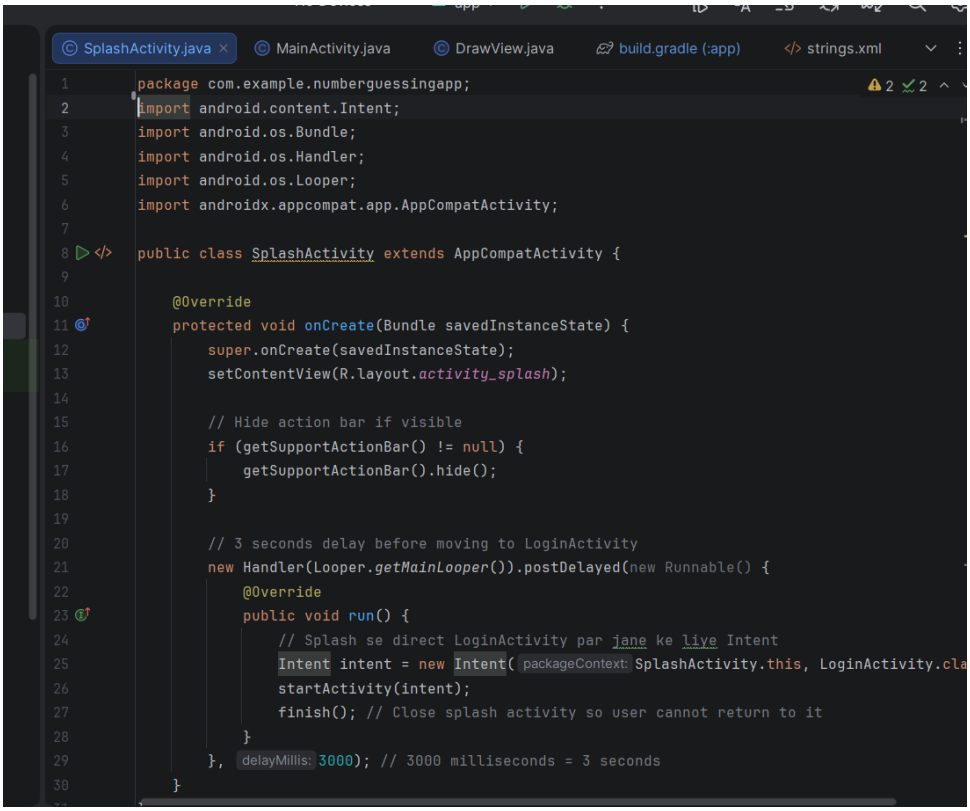
    </application>

</manifest>
```

Explanation:

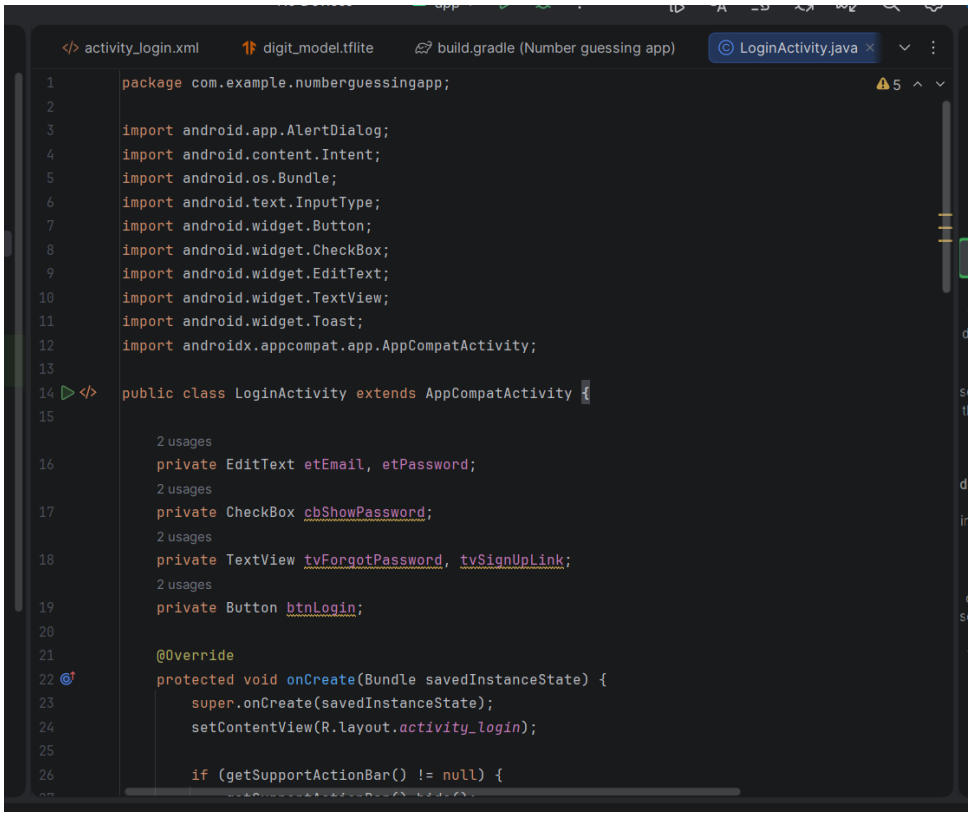
The Android Manifest file contains essential application configuration, including activity declarations, application settings, and required permissions. It helps Android identify and launch the different activities of the application.

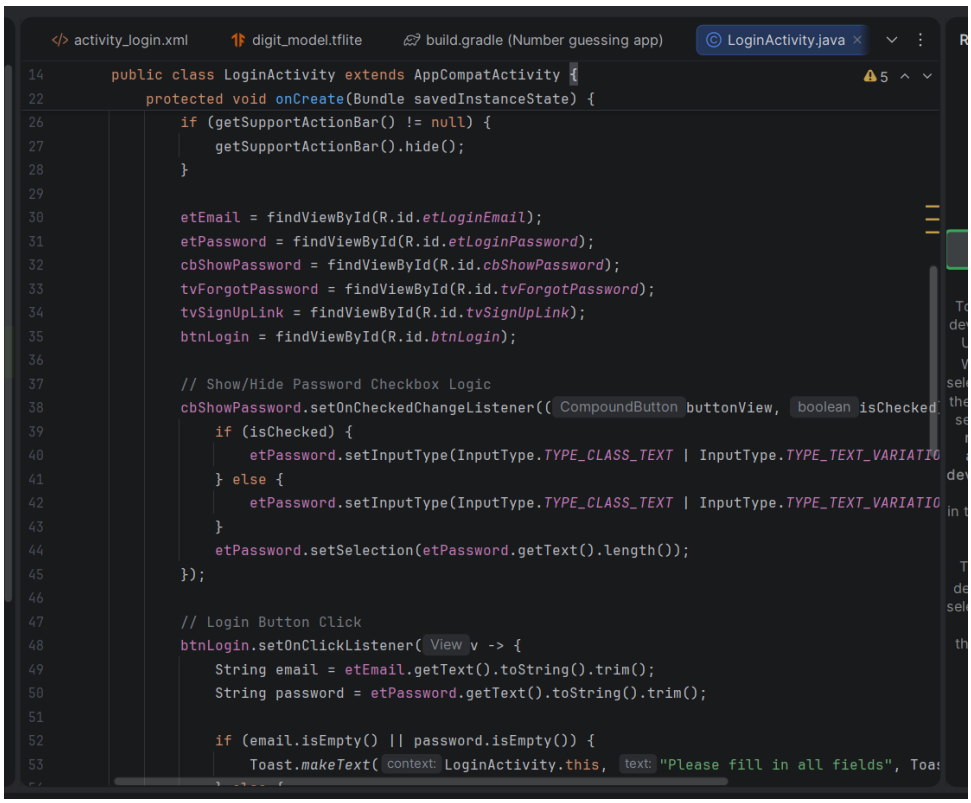
1.4 Splash Screen Implementation

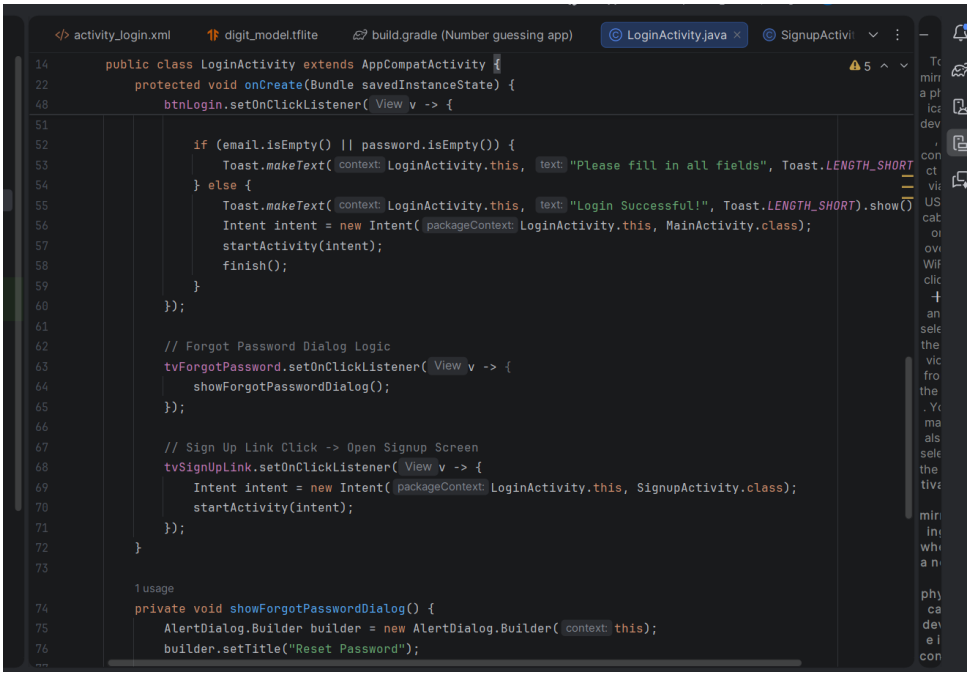


Explanation: The Splash Activity displays the initial screen of the Number Guessing App when the application starts. It provides the application's starting interface and manages navigation to the next activity.

1.5 User Login Functionality

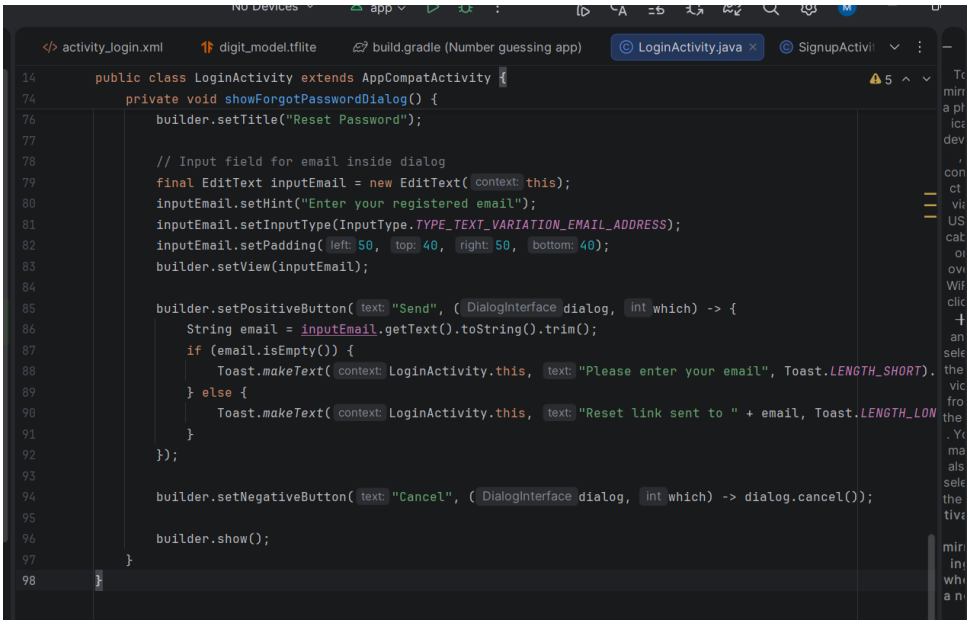






```
activity_login.xml  digit_model.tflite  build.gradle (Number guessing app)  LoginActivity.java  SignupActivi

14      public class LoginActivity extends AppCompatActivity {                                    5
22          protected void onCreate(Bundle savedInstanceState) {
48              btnLogin.setOnClickListener( View v -> {
51
52                  if (email.isEmpty() || password.isEmpty()) {
53                      Toast.makeText( context: LoginActivity.this,  text: "Please fill in all fields", Toast.LENGTH_SHORT
54                  } else {
55                      Toast.makeText( context: LoginActivity.this,  text: "Login Successful!", Toast.LENGTH_SHORT).show()
56                      Intent intent = new Intent( packageContext: LoginActivity.this, MainActivity.class);
57                      startActivity(intent);
58                      finish();
59                  }
60              });
61
62              // Forgot Password Dialog Logic
63              tvForgotPassword.setOnClickListener( View v -> {
64                  showForgotPasswordDialog();
65              });
66
67              // Sign Up Link Click -> Open Signup Screen
68              tvSignUpLink.setOnClickListener( View v -> {
69                  Intent intent = new Intent( packageContext: LoginActivity.this, SignupActivity.class);
70                  startActivity(intent);
71              });
72          }
73
              1 usage
74          private void showForgotPasswordDialog() {
75              AlertDialog.Builder builder = new AlertDialog.Builder( context: this);
76              builder.setTitle("Reset Password");
```

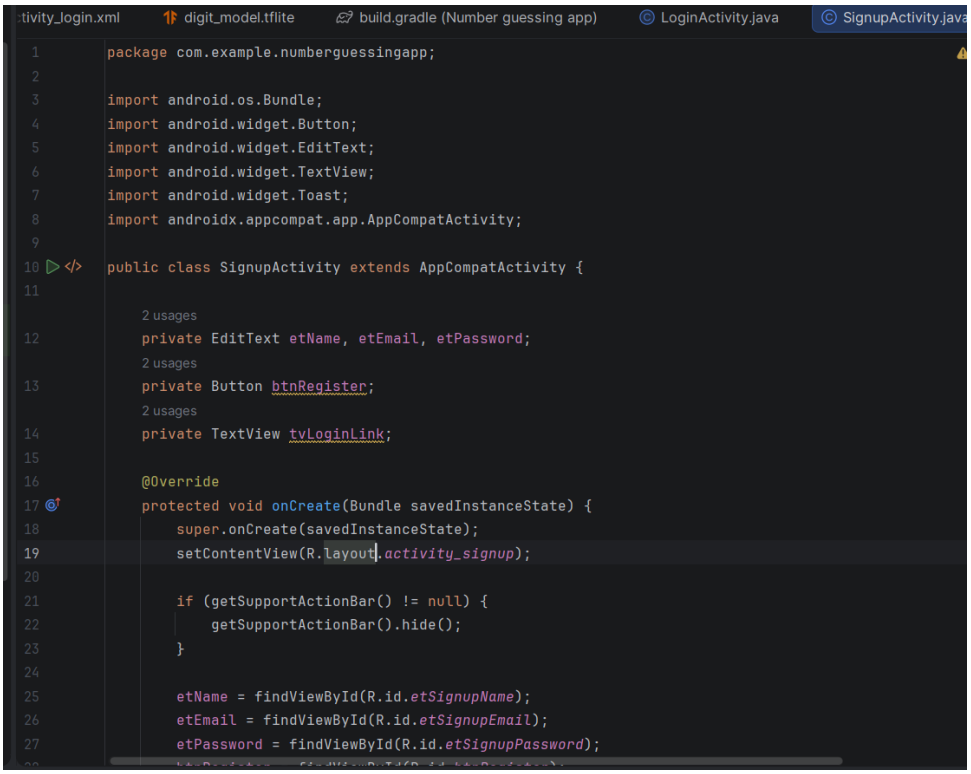


```
activity_login.xml  digit_model.tflite  build.gradle (Number guessing app)  LoginActivity.java  SignupActivi

14      public class LoginActivity extends AppCompatActivity {                                    5
74          private void showForgotPasswordDialog() {
76              builder.setTitle("Reset Password");
77
78              // Input field for email inside dialog
79              final EditText inputEmail = new EditText( context: this);
80              inputEmail.setHint("Enter your registered email");
81              inputEmail.setInputType(InputType.TYPE_TEXT_VARIATION_EMAIL_ADDRESS);
82              inputEmail.setPadding( left: 50,  top: 40,  right: 50,  bottom: 40);
83              builder.setView(inputEmail);
84
85              builder.setPositiveButton( text: "Send", ( DialogInterface dialog,  int which) -> {
86                  String email = inputEmail.getText().toString().trim();
87                  if (email.isEmpty()) {
88                      Toast.makeText( context: LoginActivity.this,  text: "Please enter your email", Toast.LENGTH_SHORT).
89                  } else {
90                      Toast.makeText( context: LoginActivity.this,  text: "Reset link sent to " + email, Toast.LENGTH_LON
91                  }
92              });
93
94              builder.setNegativeButton( text: "Cancel", ( DialogInterface dialog,  int which) -> dialog.cancel());
95
96              builder.show();
97          }
98      }
```

Explanation: The Login Activity provides an interface for existing users to access the application. It handles user input and performs the login process according to the application's authentication implementation.

1.6 User Signup Functionality



```
package com.example.numberguessingapp;

import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

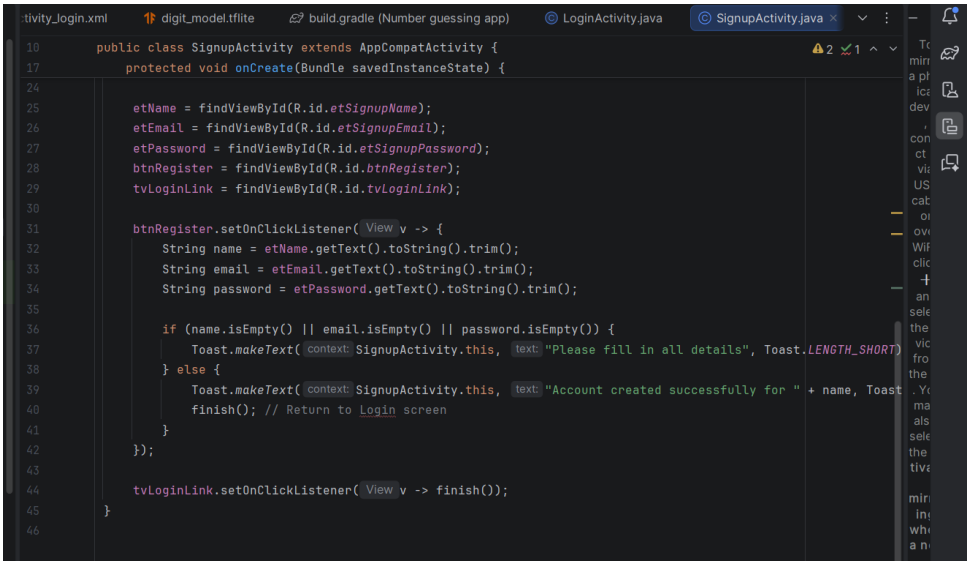
public class SignupActivity extends AppCompatActivity {

    2 usages
    private EditText etName, etEmail, etPassword;
    2 usages
    private Button btnRegister;
    2 usages
    private TextView tvLoginLink;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_signup);

        if (getSupportActionBar() != null) {
            getSupportActionBar().hide();
        }

        etName = findViewById(R.id.etSignupName);
        etEmail = findViewById(R.id.etSignupEmail);
        etPassword = findViewById(R.id.etSignupPassword);
```



```
public class SignupActivity extends AppCompatActivity {
    protected void onCreate(Bundle savedInstanceState) {

        etName = findViewById(R.id.etSignupName);
        etEmail = findViewById(R.id.etSignupEmail);
        etPassword = findViewById(R.id.etSignupPassword);
        btnRegister = findViewById(R.id.btnRegister);
        tvLoginLink = findViewById(R.id.tvLoginLink);

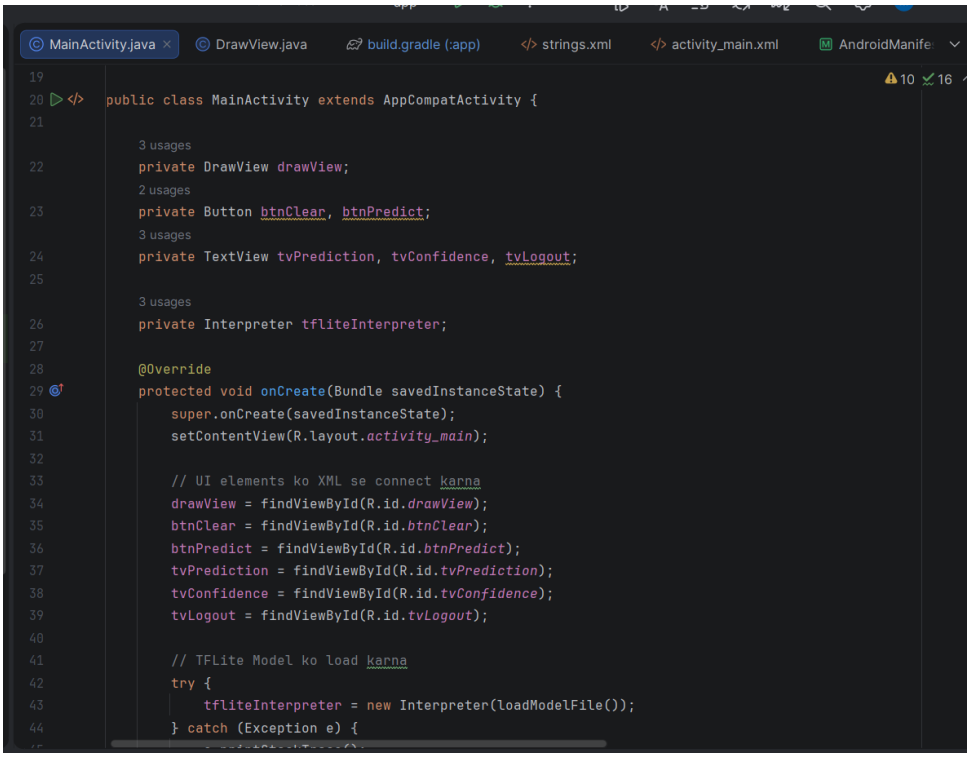
        btnRegister.setOnClickListener(View v -> {
            String name = etName.getText().toString().trim();
            String email = etEmail.getText().toString().trim();
            String password = etPassword.getText().toString().trim();

            if (name.isEmpty() || email.isEmpty() || password.isEmpty()) {
                Toast.makeText( context: SignupActivity.this,  text: "Please fill in all details", Toast.LENGTH_SHORT)
            } else {
                Toast.makeText( context: SignupActivity.this,  text: "Account created successfully for " + name, Toast
                finish(); // Return to Login screen
            }
        });

        tvLoginLink.setOnClickListener(View v -> finish());
    }
```

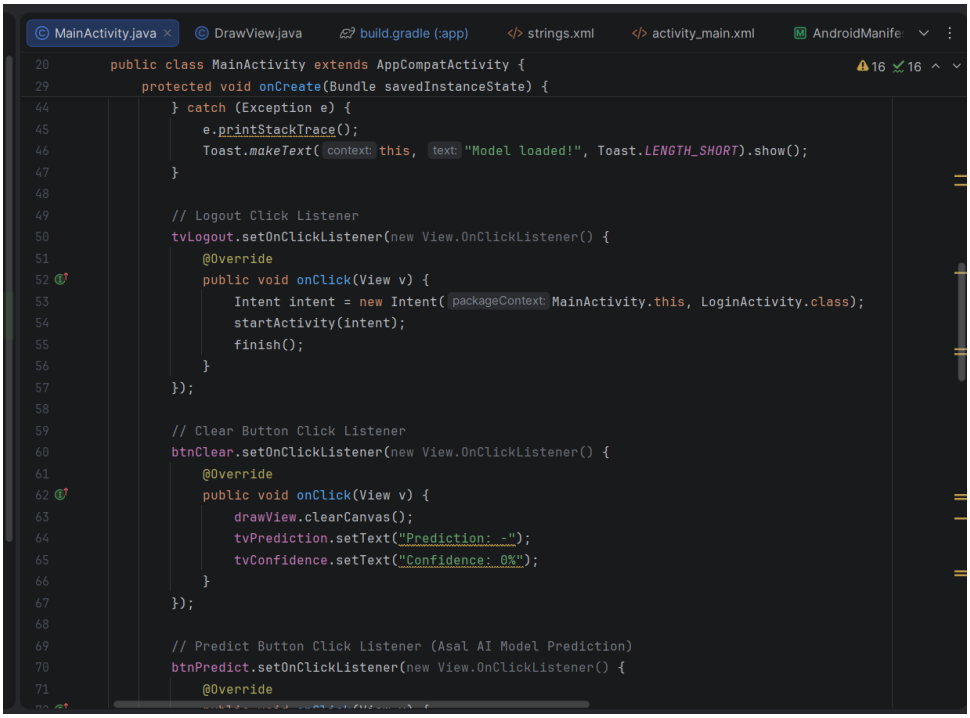
Explanation: The Signup Activity allows new users to register in the application. It collects the required user information and handles the registration process.

1.7 Main Activity Initialization



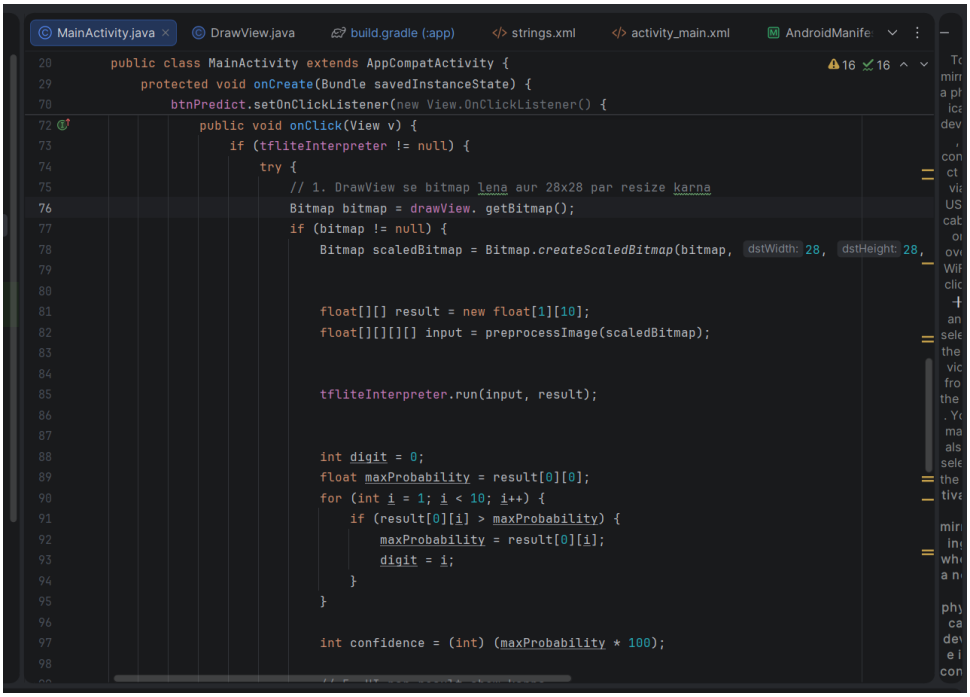
```
MainActivity.java | DrawView.java | build.gradle (:app) | strings.xml | activity_main.xml | AndroidManife

19
20  public class MainActivity extends AppCompatActivity {
21
        3 usages
22      private DrawView drawView;
        2 usages
23      private Button btnClear, btnPredict;
        3 usages
24      private TextView tvPrediction, tvConfidence, tvLogout;
25
        3 usages
26      private Interpreter tfliteInterpreter;
27
28      @Override
29      protected void onCreate(Bundle savedInstanceState) {
30          super.onCreate(savedInstanceState);
31          setContentView(R.layout.activity_main);
32
33          // UI elements ko XML se connect karna
34          drawView = findViewById(R.id.drawView);
35          btnClear = findViewById(R.id.btnClear);
36          btnPredict = findViewById(R.id.btnPredict);
37          tvPrediction = findViewById(R.id.tvPrediction);
38          tvConfidence = findViewById(R.id.tvConfidence);
39          tvLogout = findViewById(R.id.tvLogout);
40
41          // TFLite Model ko load karna
42          try {
43              tfliteInterpreter = new Interpreter(loadModelFile());
44          } catch (Exception e) {
```



```
MainActivity.java | DrawView.java | build.gradle (:app) | strings.xml | activity_main.xml | AndroidManife

20  public class MainActivity extends AppCompatActivity {
29      protected void onCreate(Bundle savedInstanceState) {
44          } catch (Exception e) {
45              e.printStackTrace();
46              Toast.makeText( context: this,  text: "Model loaded!", Toast.LENGTH_SHORT).show();
47          }
48
49          // Logout Click Listener
50          tvLogout.setOnClickListener(new View.OnClickListener() {
51              @Override
52              public void onClick(View v) {
53                  Intent intent = new Intent( packageContext: MainActivity.this, LoginActivity.class);
54                  startActivity(intent);
55                  finish();
56              }
57          });
58
59          // Clear Button Click Listener
60          btnClear.setOnClickListener(new View.OnClickListener() {
61              @Override
62              public void onClick(View v) {
63                  drawView.clearCanvas();
64                  tvPrediction.setText("Prediction: -");
65                  tvConfidence.setText("Confidence: 0%");
66              }
67          });
68
69          // Predict Button Click Listener (Asal AI Model Prediction)
70          btnPredict.setOnClickListener(new View.OnClickListener() {
71              @Override
```


MainActivity.java
DrawView.java
build.gradle (:app)
strings.xml
activity_main.xml
AndroidManife
16 16
public class MainActivity extends AppCompatActivity {
protected void onCreate(Bundle savedInstanceState) {
btnPredict.setOnClickListener(new View.OnClickListener() {
public void onClick(View v) {
if (tfliteInterpreter != null) {
try {
// 1. DrawView se bitmap lena aur 28x28 par resize karna
Bitmap bitmap = drawView. getBitmap();
if (bitmap != null) {
Bitmap scaledBitmap = Bitmap.createScaledBitmap(bitmap, dstWidth: 28, dstHeight: 28,
float[][] result = new float[1][10];
float[][][][] input = preprocessImage(scaledBitmap);
tfliteInterpreter.run(input, result);
int digit = 0;
float maxProbability = result[0][0];
for (int i = 1; i < 10; i++) {
if (result[0][i] > maxProbability) {
maxProbability = result[0][i];
digit = i;
}
}
int confidence = (int) (maxProbability * 100);


btnPredict.setOnClickListener(new View.OnClickListener() {
public void onClick(View v) {
int confidence = (int) (maxProbability * 100);
// 5. UI par result show karna
tvPrediction.setText("Prediction: " + digit);
tvConfidence.setText("Confidence: " + confidence + "%");
} else {
Toast.makeText(context: MainActivity.this, text: "Firstly draw something!", Toast.LE
}
} catch (Exception e) {
e.printStackTrace();
Toast.makeText(context: MainActivity.this, text: "There is an error in predection!", Toa
}
} else {
Toast.makeText(context: MainActivity.this, text: "model is not initializing!", Toast.LENGTH_
}
}
});
}
// Assets folder se .tflite file read karne ka method
1 usage
private MappedByteBuffer loadModelFile() throws IOException {
AssetFileDescriptor fileDescriptor = getAssets().openFd(fileName: "digit_model.tflite"); // Agar aap ki
FileInputStream inputStream = new FileInputStream(fileDescriptor.getFileDescriptor());
FileChannel fileChannel = inputStream.getChannel();
long startOffset = fileDescriptor.getStartOffset();

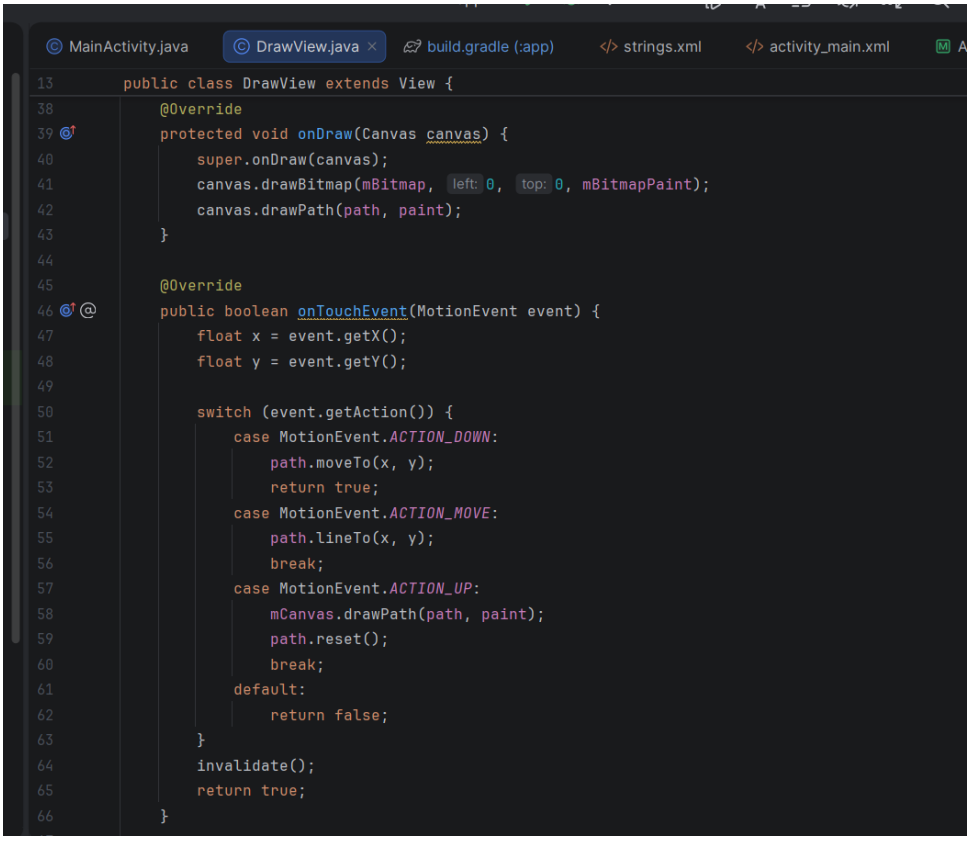
```
117     private MappedByteBuffer loadModelFile() throws IOException {
118         AssetFileDescriptor fileDescriptor = getAssets().openFd( fileName: "digit_model.tflite"); // Agar
119         FileInputStream inputStream = new FileInputStream(fileDescriptor.getFileDescriptor());
120         FileChannel fileChannel = inputStream.getChannel();
121         long startOffset = fileDescriptor.getStartOffset();
122         long declaredLength = fileDescriptor.getDeclaredLength();
123         return fileChannel.map(FileChannel.MapMode.READ_ONLY, startOffset, declaredLength);
124     }
125
126     // Bitmap ko model ke input format (28x28 grayscale normalized) mein convert karna
        1 usage
127 @   private float[][][][] preprocessImage(Bitmap bitmap) {
128         float[][][][] inputArray = new float[1][28][28][1];
129         int[] pixels = new int[28 * 28];
130         bitmap.getPixels(pixels,  offset: 0,  stride: 28,  x: 0,  y: 0,  width: 28,  height: 28);
131
132         for (int i = 0; i < 28; i++) {
133             for (int j = 0; j < 28; j++) {
134                 int pixel = pixels[i * 28 + j];
135                 // Background black (0) aur drawing white (255) hone ki normalization
136                 float red = (pixel >> 16) & 0xFF;
137                 float green = (pixel >> 8) & 0xFF;
138                 float blue = pixel & 0xFF;
139                 float gray = (red + green + blue) / 3.0f / 255.0f;
140                 inputArray[0][i][j][0] = gray;
141             }
142         }
143         return inputArray;
144     }
```

Explanation: The Main Activity controls the primary functionality of the Number Guessing App. It connects the user interface with the digit recognition functionality and manages user interactions.

1.8 Drawing View Implementation

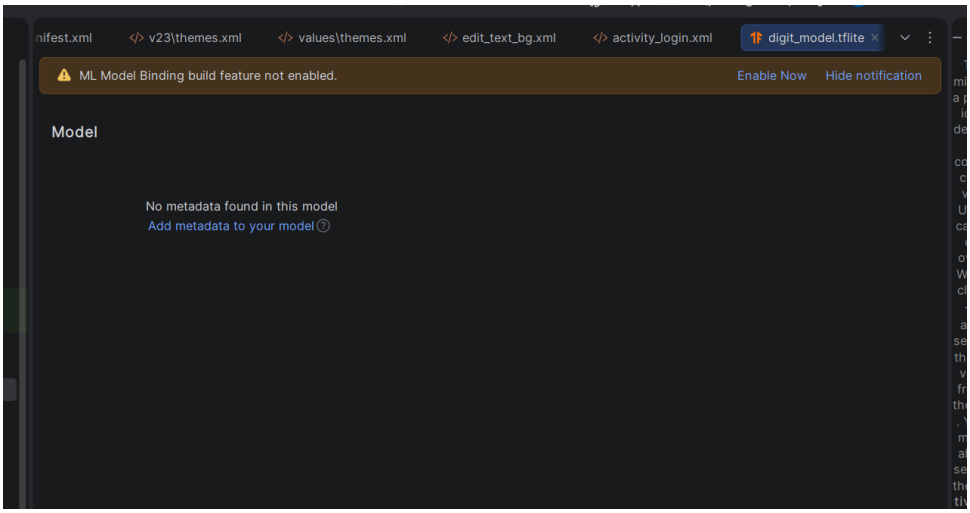
```
MainActivity.java | DrawView.java | build.gradle (:app) | strings.xml | activity_main.xml

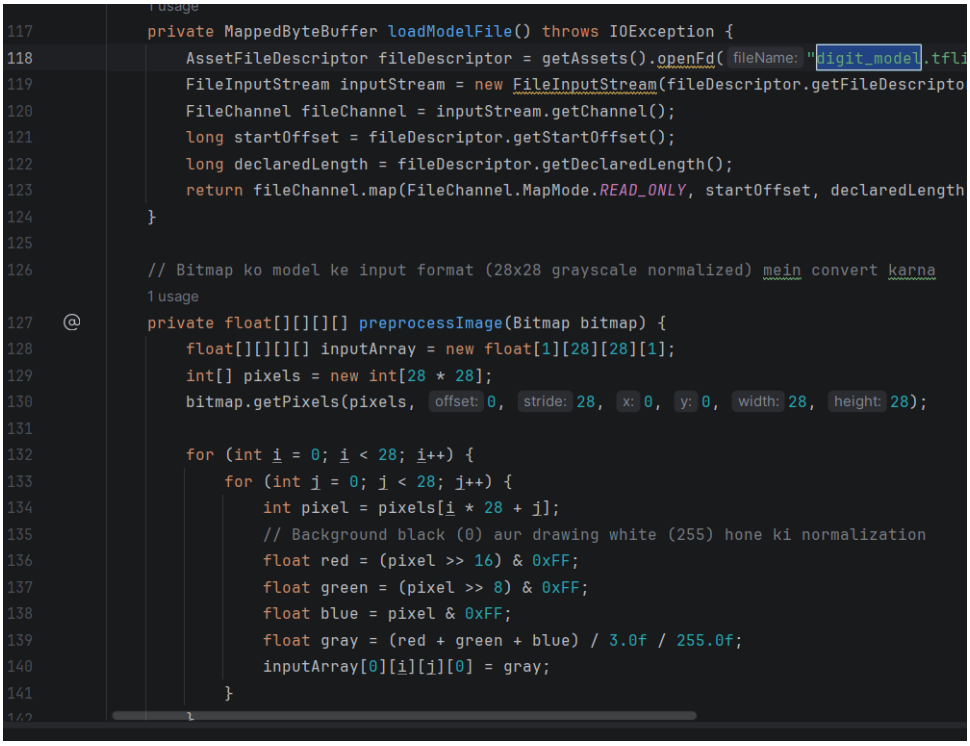
13  public class DrawView extends View {
        7 usages
14      private Path path = new Path();
        9 usages
15      private Paint paint = new Paint();
        4 usages
16      private Bitmap mBitmap;
        4 usages
17      private Canvas mCanvas;
        1 usage
18      private Paint mBitmapPaint = new Paint(Paint.DITHER_FLAG);
19
20      public DrawView(Context context, AttributeSet attrs) {
21          super(context, attrs);
22          paint.setAntiAlias(true);
23          paint.setColor(Color.WHITE); // Black background par white color se likhne ke liye
24          paint.setStyle(Paint.Style.STROKE);
25          paint.setStrokeJoin(Paint.Join.ROUND);
26          paint.setStrokeCap(Paint.Cap.ROUND);
27          paint.setStrokeWidth(45f); // Brush ki motapa (thickness)
28      }
29
30      @Override
31      protected void onSizeChanged(int w, int h, int oldw, int oldh) {
32          super.onSizeChanged(w, h, oldw, oldh);
33          mBitmap = Bitmap.createBitmap(w, h, Bitmap.Config.ARGB_8888);
34          mCanvas = new Canvas(mBitmap);
35          mCanvas.drawColor(Color.BLACK); // Background black set karna
36      }
```



Explanation: The DrawView class provides a drawing area where users can draw handwritten numerical digits using their fingers. It manages the drawing surface and displays the user's handwritten input.

1.9 TensorFlow Lite Model Loading





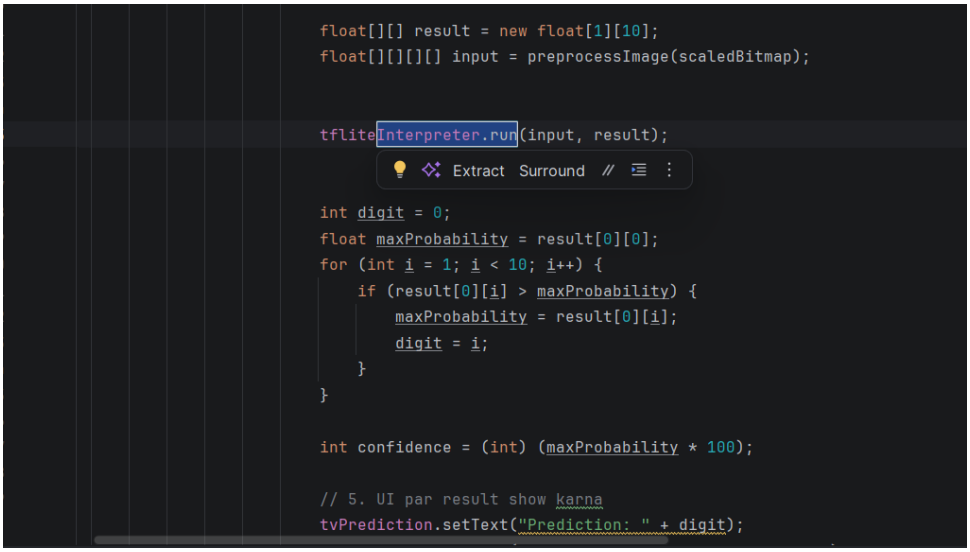
```
1 usage
private MappedByteBuffer loadModelFile() throws IOException {
    AssetFileDescriptor fileDescriptor = getAssets().openFd( fileName: "digit_model.tfli
    FileInputStream inputStream = new FileInputStream(fileDescriptor.getFileDescripto
    FileChannel fileChannel = inputStream.getChannel();
    long startOffset = fileDescriptor.getStartOffset();
    long declaredLength = fileDescriptor.getDeclaredLength();
    return fileChannel.map(FileChannel.MapMode.READ_ONLY, startOffset, declaredLength
}

// Bitmap ko model ke input format (28x28 grayscale normalized) mein convert karna
1 usage
private float[][][][] preprocessImage(Bitmap bitmap) {
    float[][][][] inputArray = new float[1][28][28][1];
    int[] pixels = new int[28 * 28];
    bitmap.getPixels(pixels,  offset: 0,  stride: 28,  x: 0,  y: 0,  width: 28,  height: 28);

    for (int i = 0; i < 28; i++) {
        for (int j = 0; j < 28; j++) {
            int pixel = pixels[i * 28 + j];
            // Background black (0) aur drawing white (255) hone ki normalization
            float red = (pixel >> 16) & 0xFF;
            float green = (pixel >> 8) & 0xFF;
            float blue = pixel & 0xFF;
            float gray = (red + green + blue) / 3.0f / 255.0f;
            inputArray[0][i][j][0] = gray;
        }
```

Explanation: The application uses the trained TensorFlow Lite model stored in the assets folder. The model is loaded into memory so that it can perform handwritten digit recognition directly on the Android device.

1.10 Model Inference and Prediction



```
float[][] result = new float[1][10];
float[][][][] input = preprocessImage(scaledBitmap);

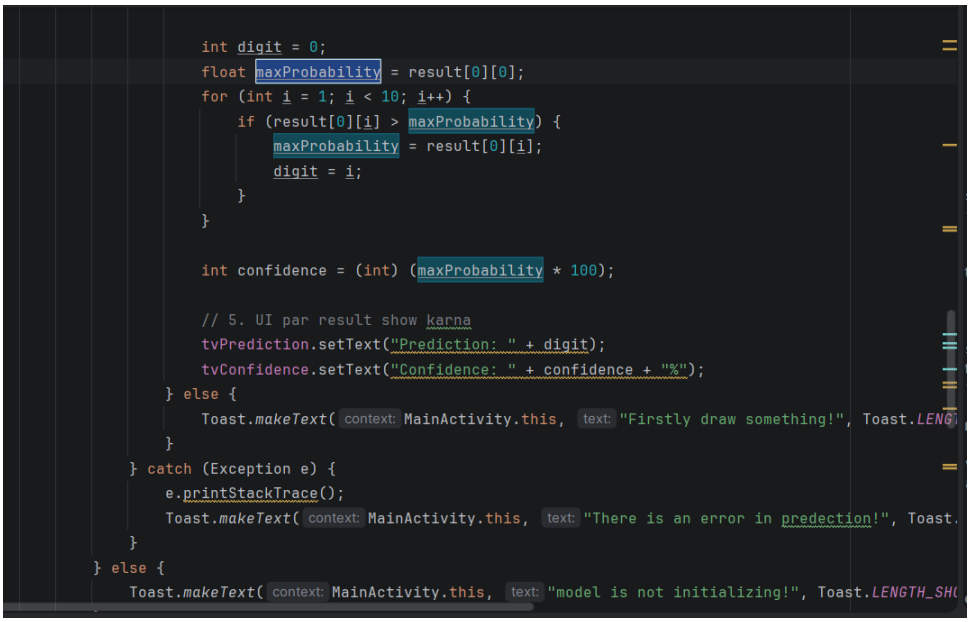

tfliteInterpreter.run(input, result);
        Extract  Surround  //
int digit = 0;
float maxProbability = result[0][0];
for (int i = 1; i < 10; i++) {
    if (result[0][i] > maxProbability) {
        maxProbability = result[0][i];
        digit = i;
    }
}

int confidence = (int) (maxProbability * 100);

// 5. UI par result show karna
tvPrediction.setText("Prediction: " + digit);
```

Explanation: This code passes the prepared input data to the TensorFlow Lite Interpreter. The model processes the input and generates prediction scores for the possible numerical digits.

1.11 Output Processing



```
            int digit = 0;
            float maxProbability = result[0][0];
            for (int i = 1; i < 10; i++) {
                if (result[0][i] > maxProbability) {
                    maxProbability = result[0][i];
                    digit = i;
                }
            }

            int confidence = (int) (maxProbability * 100);

            // 5. UI par result show karna
            tvPrediction.setText("Prediction: " + digit);
            tvConfidence.setText("Confidence: " + confidence + "%");
        } else {
            Toast.makeText( context: MainActivity.this, text: "Firstly draw something!", Toast.LENG
        }
    } catch (Exception e) {
        e.printStackTrace();
        Toast.makeText( context: MainActivity.this, text: "There is an error in predection!", Toast.
    }
} else {
    Toast.makeText( context: MainActivity.this, text: "model is not initializing!", Toast.LENGTH_SH
```

Explanation: This code processes the output generated by the TensorFlow Lite model. It compares the prediction scores and identifies the digit with the highest probability as the predicted result.

2 Conclusion

The Number Guessing App demonstrates the integration of a Deep Learning model into an Android application using TensorFlow Lite. The code snippets explain the important components of the application, including Gradle configuration, user authentication, drawing functionality, model loading.

The project provides practical understanding of how handwritten digit recognition can be implemented through on-device machine learning. It also demonstrates how Android development and TensorFlow Lite can be combined to create an interactive AI-based mobile application.